PASCUAL BRAVO



Técnicas de resolución de problemas en desarrollo de software

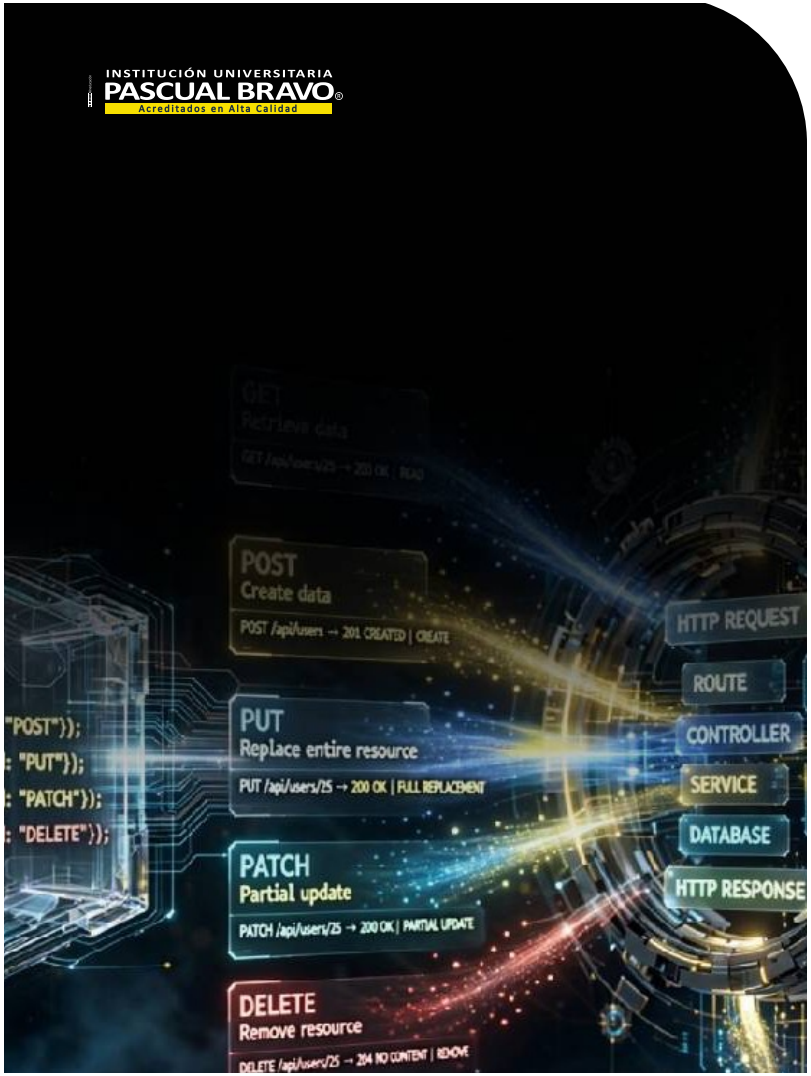
INSTITUCIÓN UNIVERSITARIA
PASCUAL BRAVO
Acreditados en Alta Calidad

GET, POST, PUT, PATCH y DELETE



Métodos HTTP utilizados para comunicarnos entre un frontend y un backend/API o entre sistemas

Frontend
|
| GET /api/productos
▼
Backend
|
▼
Base de datos



Los métodos indican qué queremos hacer con el registro.

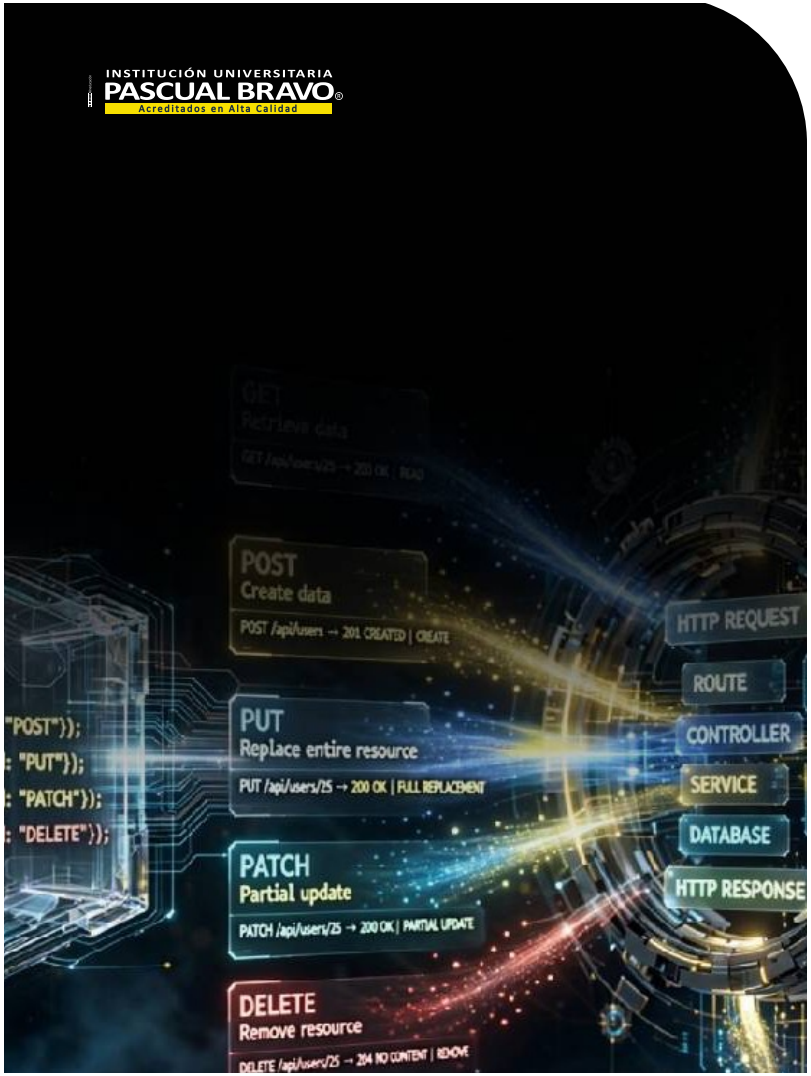
Método	Uso principal	Ejemplo
GET	Consultar	Obtener productos
POST	Crear	Crear un producto
PUT	Reemplazar/actualizar completo	Actualizar todos los datos
PATCH	Actualizar parcialmente	Cambiar solo el precio
DELETE	Eliminar	Eliminar un producto



GET — Obtener información
Se utiliza cuando queremos consultar información.

GET /api/productos

Significa: Dame los registros de los productos



El servidor podría responder:

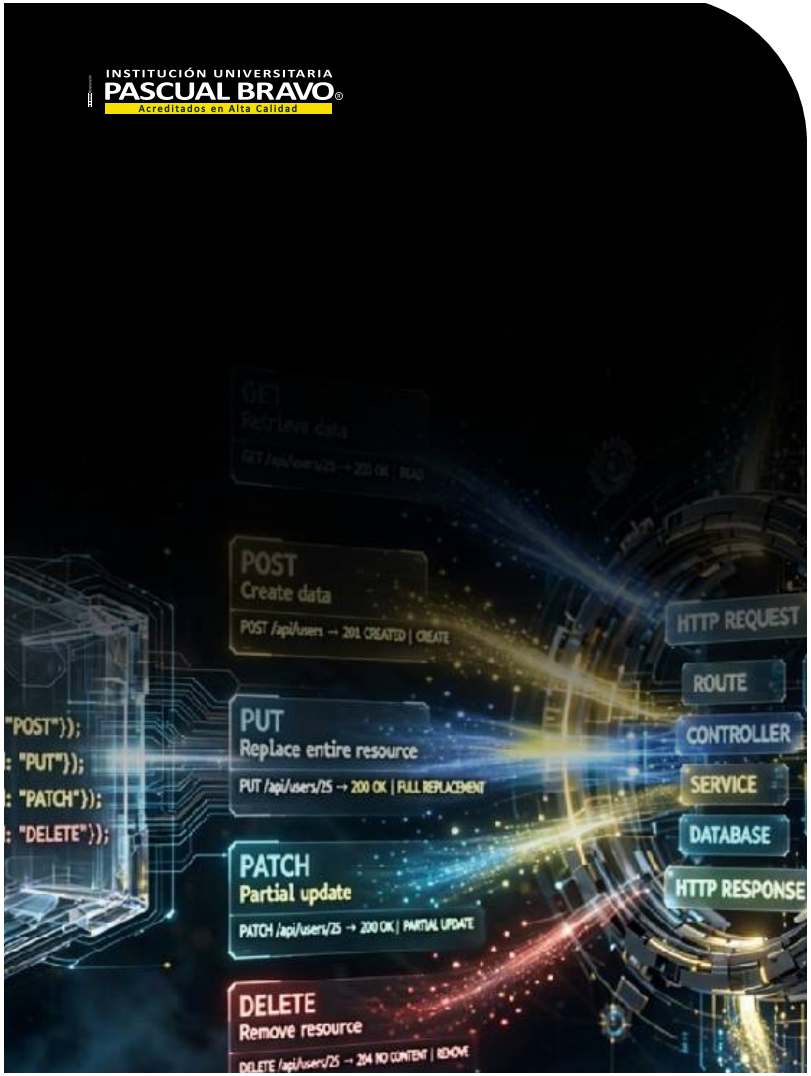
```
[
  {
    "id": 1,
    "nombre": "Mouse",
    "precio": 50000
  },
  {
    "id": 2,
    "nombre": "Teclado",
    "precio": 80000
  }
]
```



POST — Crear información
Normalmente se utiliza para crear un nuevo registro.

POST /api/productos

Significa: Envío estos datos para procesar y/o guardar

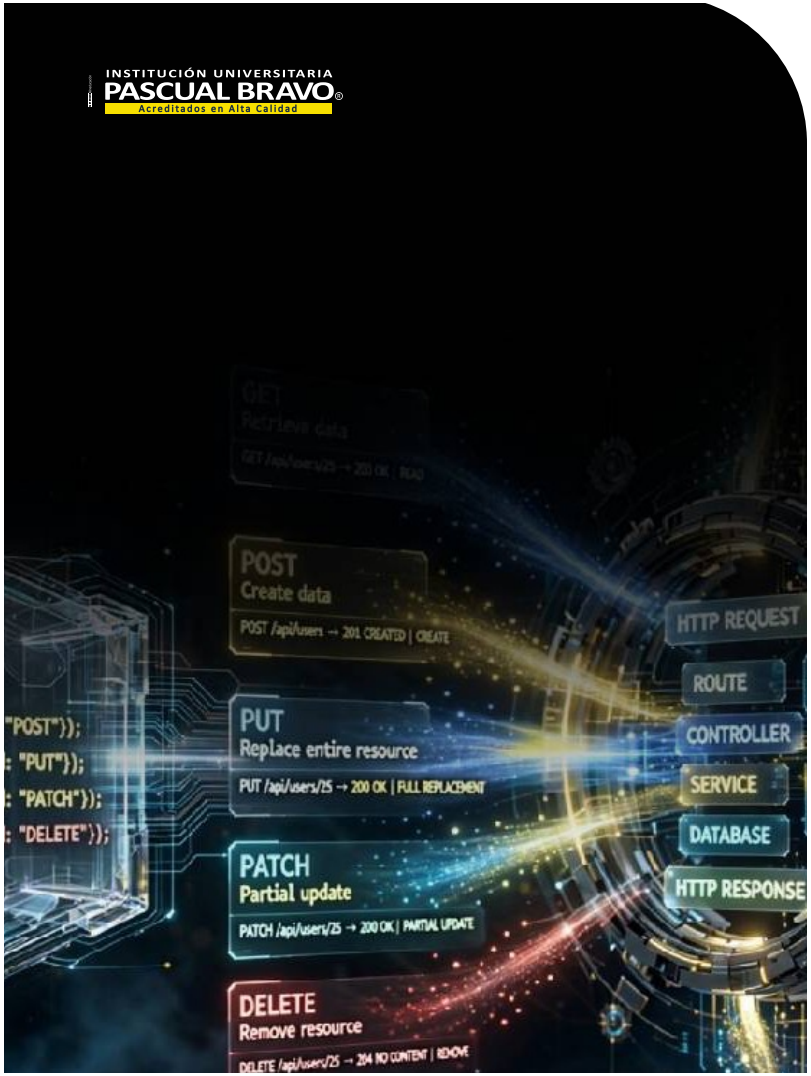


Enviamos:

```
{
  "nombre": "Monitor",
  "precio": 800000
}
```

El servidor podría crear:

```
{
  "id": 3,
  "nombre": "Monitor",
  "precio": 800000
}
```

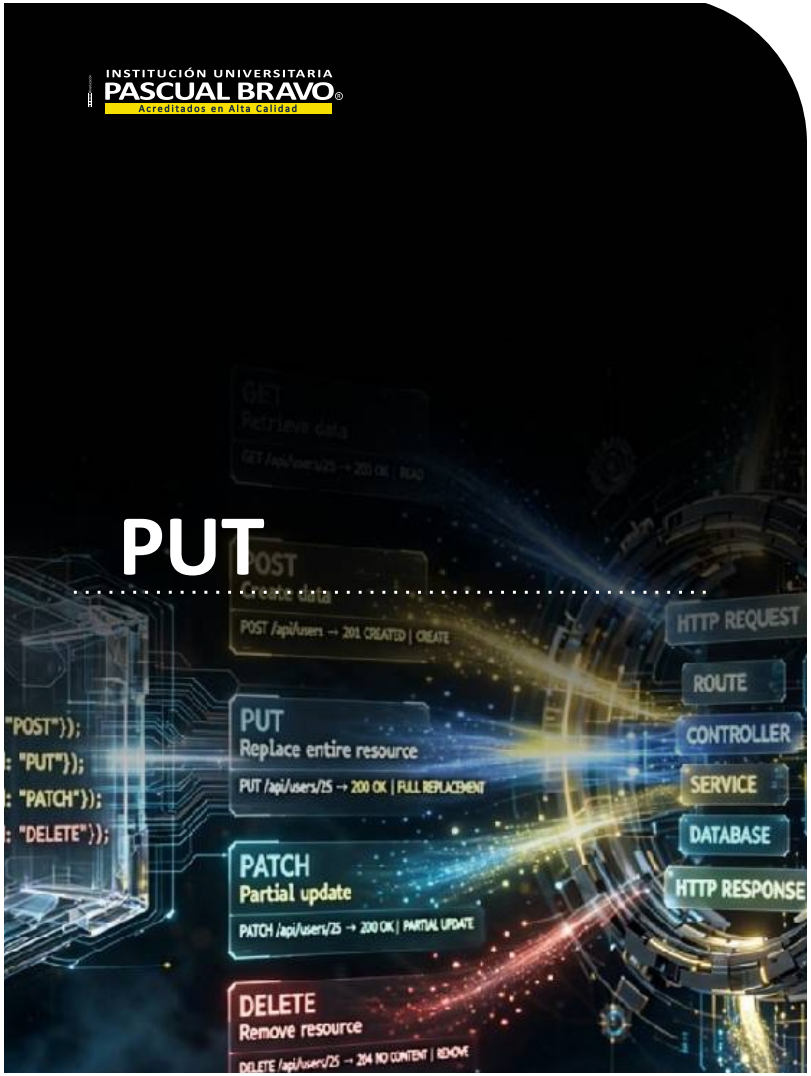


¿Dónde se utiliza?

Formulario
↓
POST /api/productos
↓
Backend
↓
INSERT INTO productos

También:

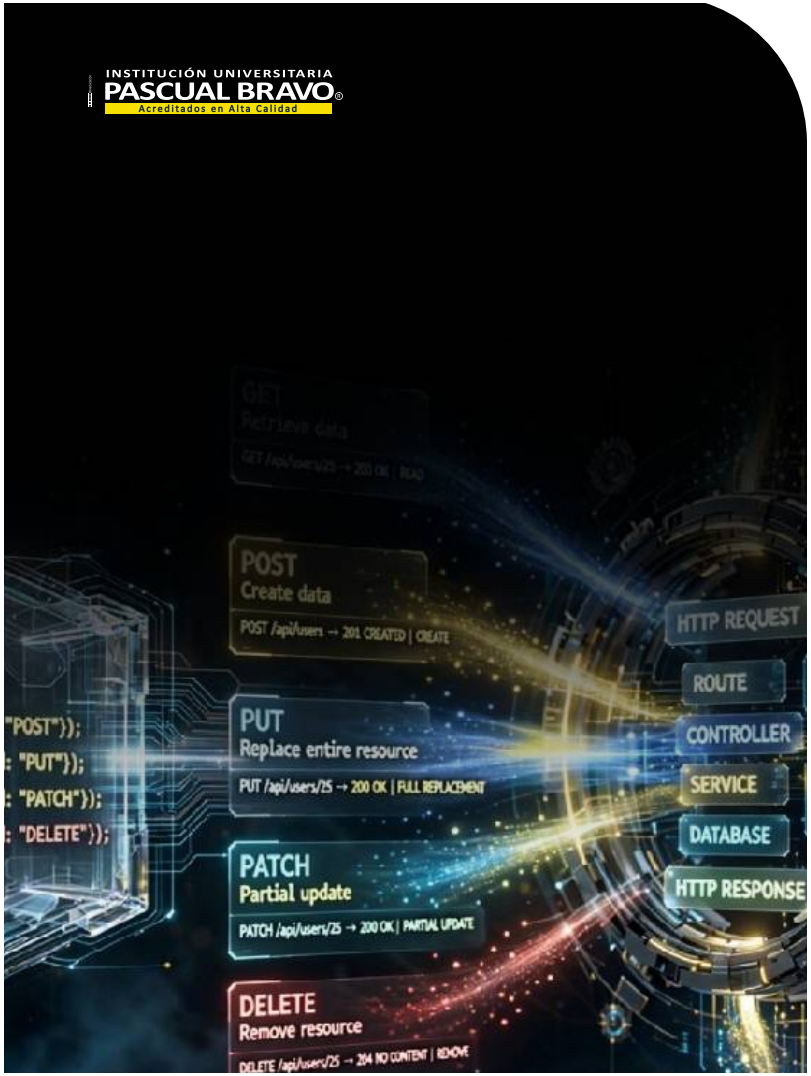
- Registrar usuarios.
- Crear pedidos.
- Crear productos.
- Crear facturas.
- Crear reservas.
- Iniciar determinados procesos del servidor.
- Entre otros



PUT — Actualizar completamente
Se utiliza normalmente para reemplazar completamente un recurso existente.

PUT /api/productos/1

Significa: Envío estos datos para reemplazar los ya existentes

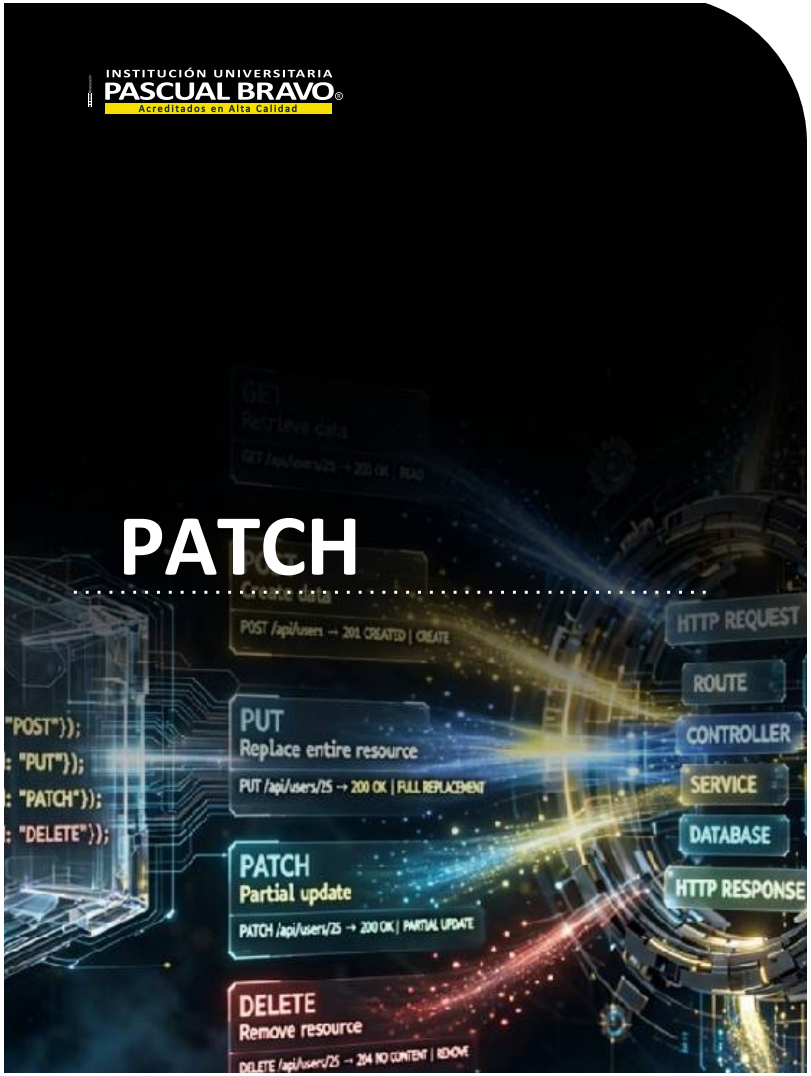


Tenemos:

```
{
  "id": 1,
  "nombre": "Mouse",
  "precio": 50000,
  "marca": "Logitech"
}
```

Enviamos:

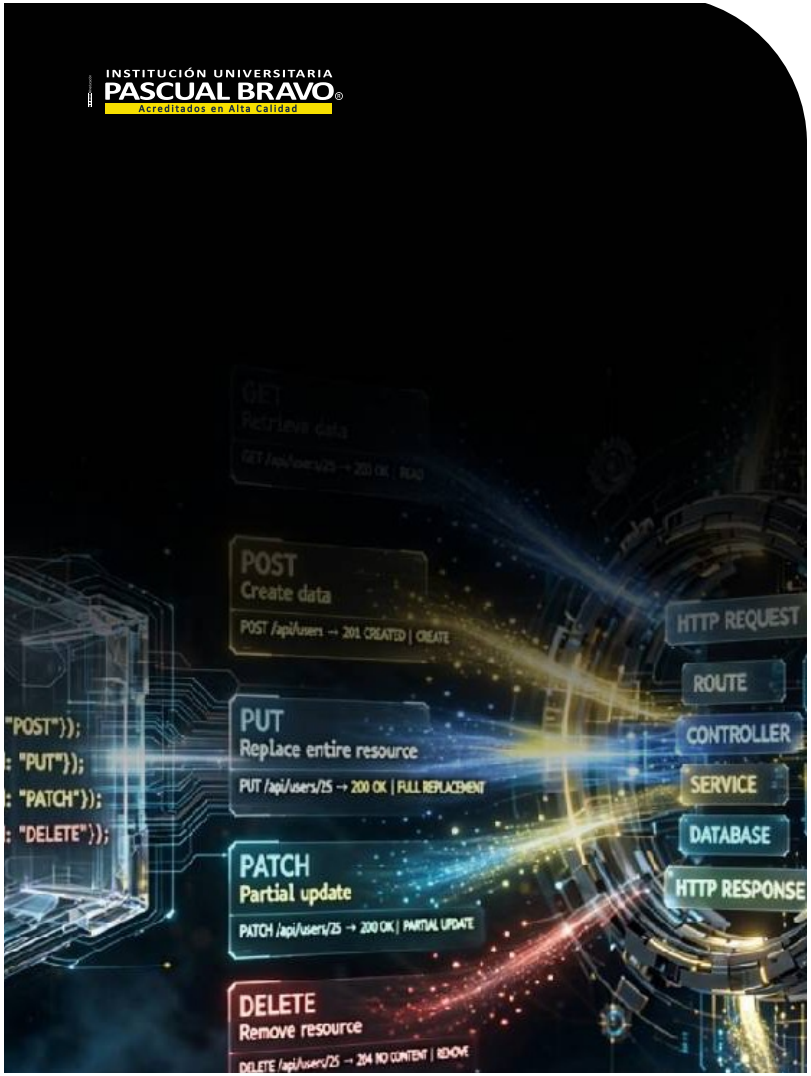
```
{
  "nombre": "Mouse inalámbrico",
  "precio": 70000,
  "marca": "Logitech"
}
```



PATCH — Actualizar parcialmente
Es parecido a PUT, pero normalmente se utiliza cuando queremos modificar **solamente una parte del registro.**

PATCH /api/productos/1

Significa: Envío esto para reemplazar el ya existente.



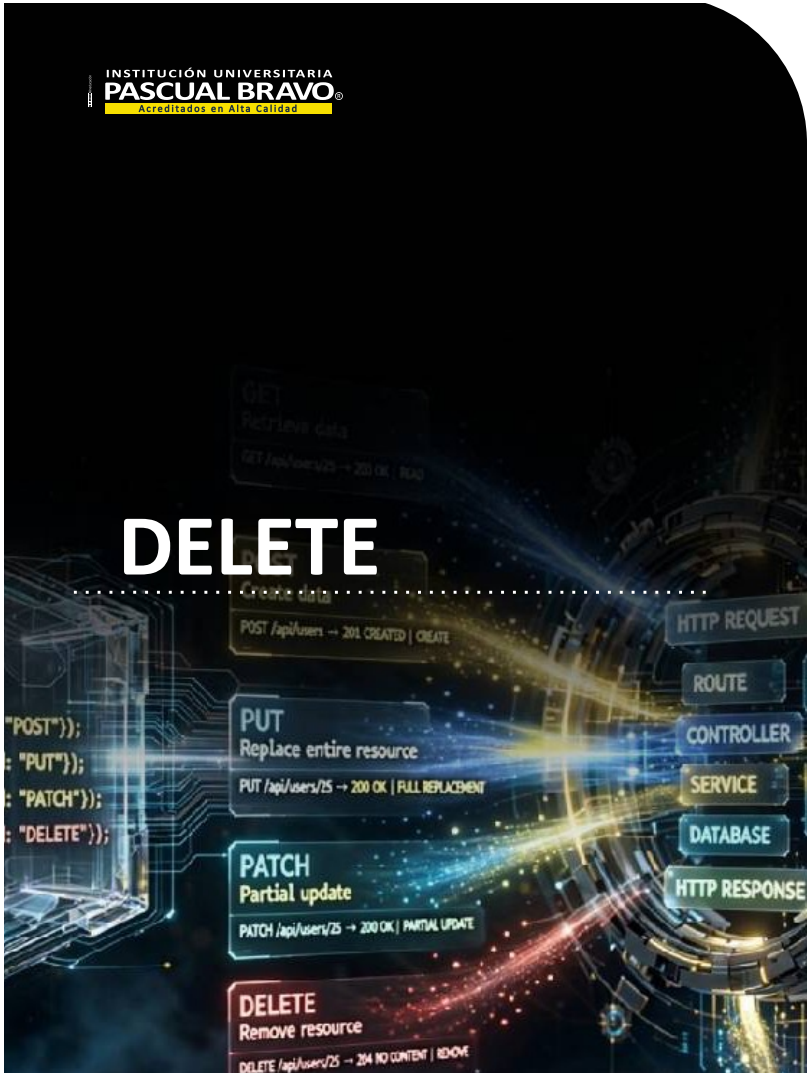
Tenemos:

```
{
  "id": 1,
  "nombre": "Mouse",
  "precio": 50000,
  "marca": "Logitech"
}
```

Enviamos:

```
{
  "precio": 60000
}
```

No necesitamos enviar todos los atributos con sus valores, solo **enviamos lo que se requiere cambiar y/o actualizar.**



DELETE — Eliminar
Se utiliza para eliminar un registro.

DELETE /api/productos/1

Significa: Elimina el registro con el id 1

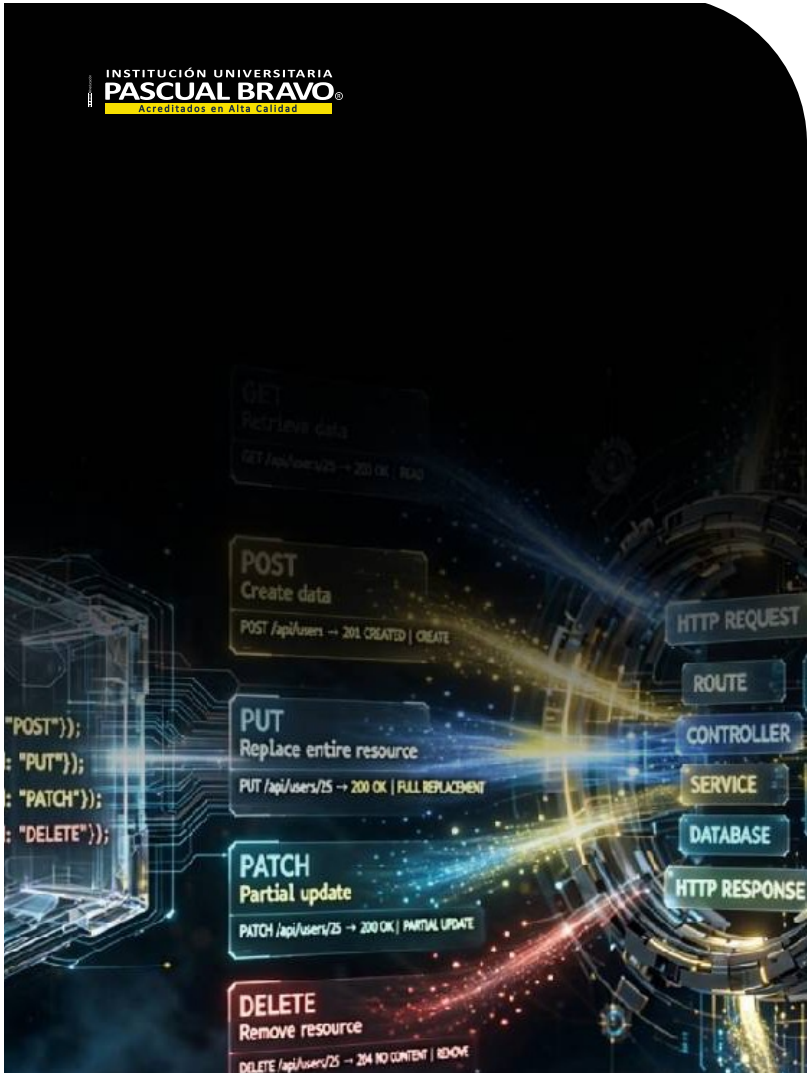
El backend podría ejecutar algo equivalente a:
DELETE FROM productos WHERE id = 1;

INSTITUCIÓN UNIVERSITARIA
PASCUAL BRAVO
Acreditados en Alta Calidad

hash de una transacción



Un hash es una representación de **tamaño fijo** generada a partir de los datos de una transacción.



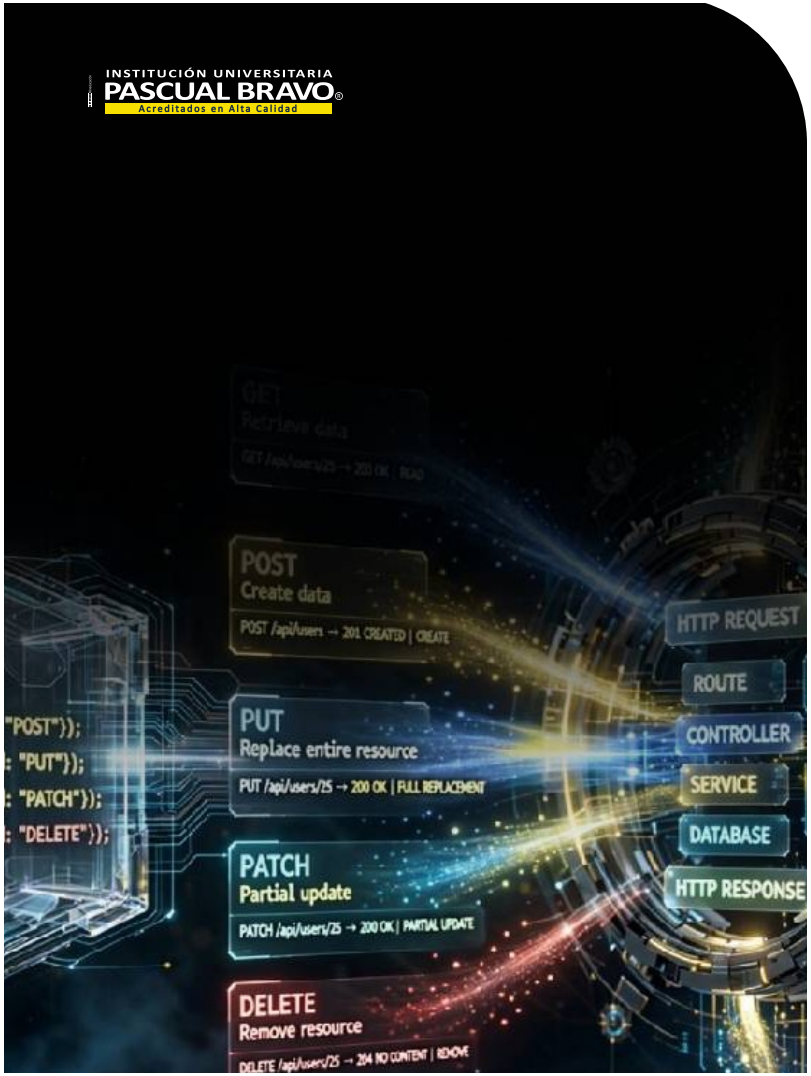
Por ejemplo:


```
{
 "id": 1001,
 "producto": "Mouse",
 "cantidad": 2,
 "valor": 50000
}
```


Podemos generar un hash SHA-256: 8f3a...c91e

Si alguien modifica la transacción, el hash será diferente.

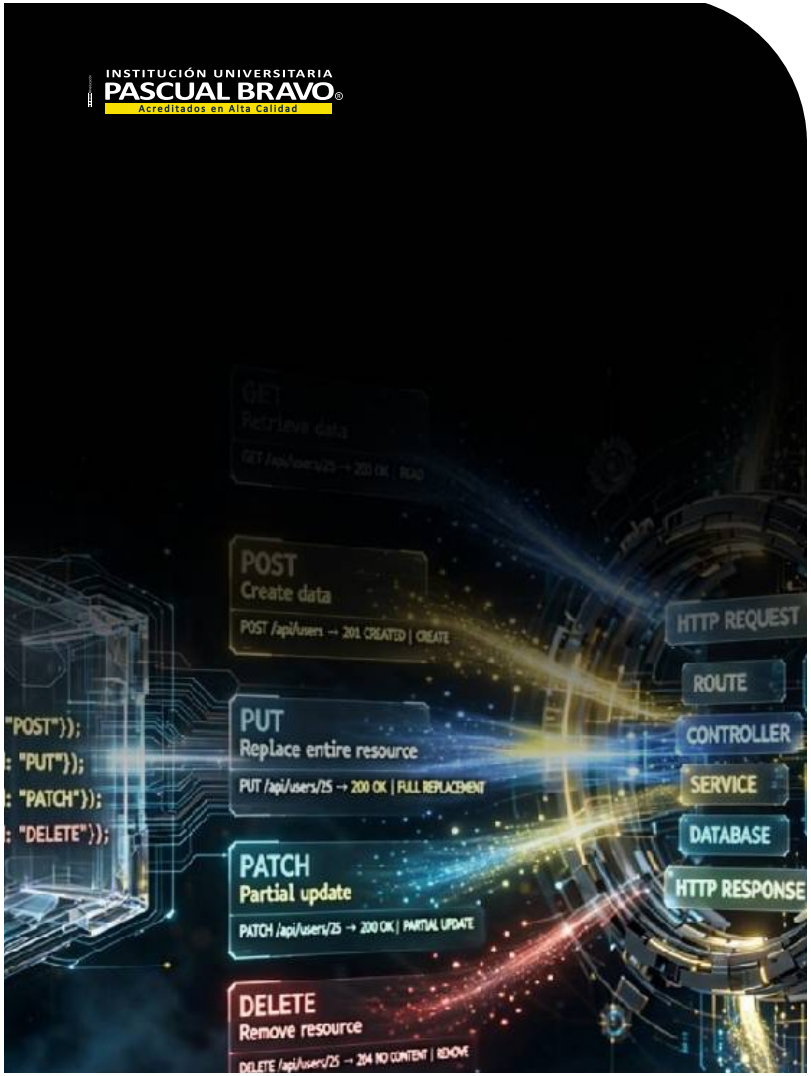

```
{
 "id": 1001,
 "producto": "Mouse",
 "cantidad": 3,
 "valor": 50000
}
```



Transacción original
↓
SHA-256
↓
8f3a...c91e

Transacción modificada
↓
SHA-256
↓
2ab7...91f4

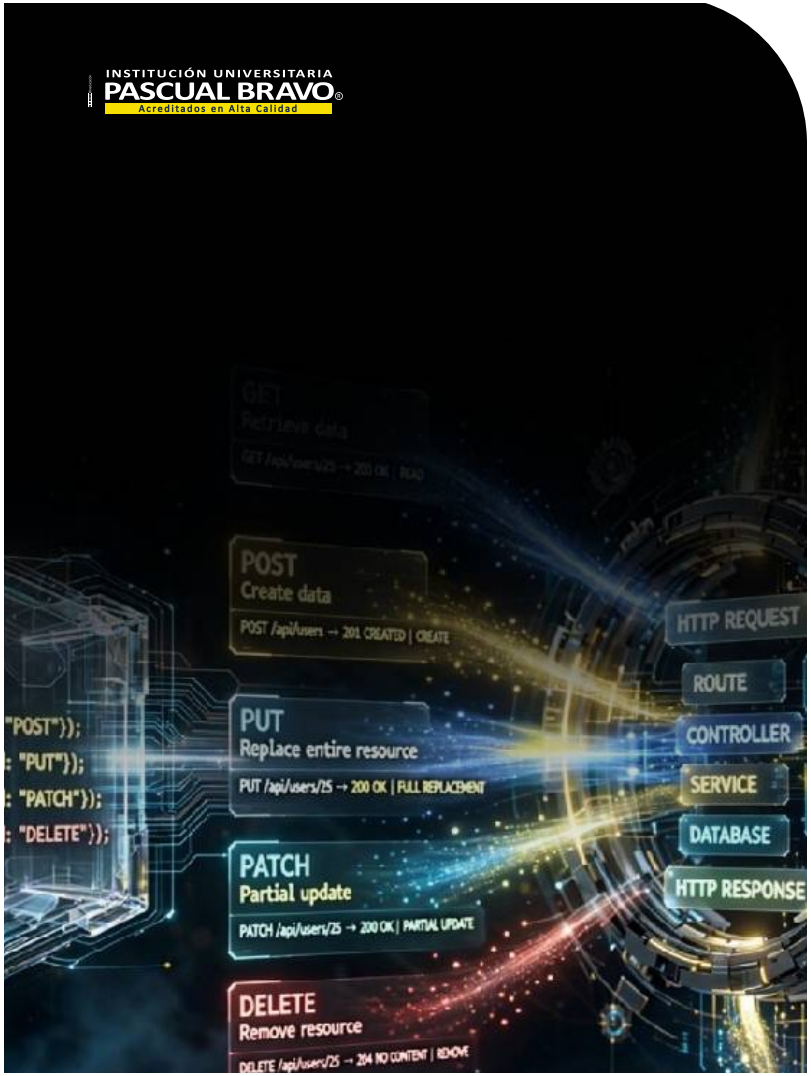
Por eso podemos detectar que los datos cambiaron.



Ejemplo aplicado a un pago

```
{
  "id_transaccion": "TX1001",
  "usuario": 25,
  "valor": 150000,
  "moneda": "COP",
  "fecha": "2026-09-25"
}
```

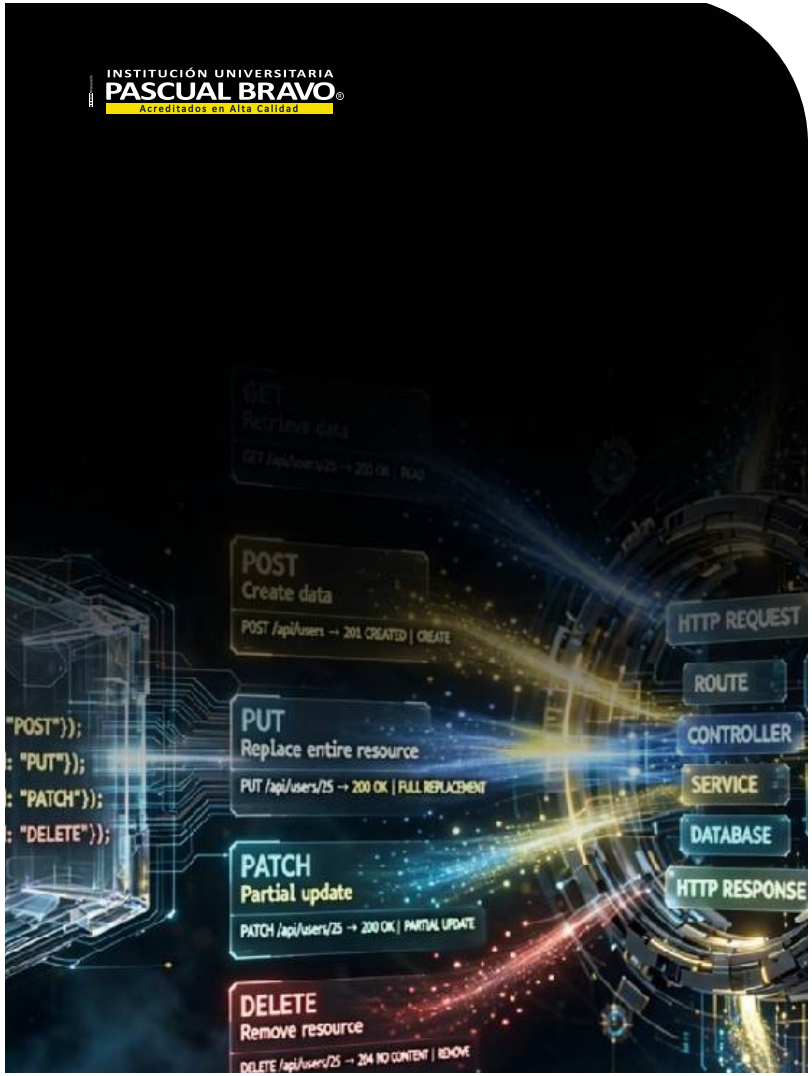
Hash = SHA256(datos)



Cuando recibimos una solicitud
Cliente
↓
Transacción
↓
Calcular SHA-256
↓
Comparar hash
↓
¿Coincide?

Sí No
↓ ↓
Aceptar Rechazar

Un hash por sí solo no autentica al remitente




```
import hashlib
import hmac
import json

LLAVE_SECRETA = b"mi_llave_privada_123"
# La b significa que estamos creando un objeto de tipo bytes, no un str normal


transaccion = {
   "id": 1001,
   "producto": "Mouse",
   "cantidad": 4,
   "valor": 50000
}

datos = json.dumps(
   transaccion,
   sort_keys=True,
   separators=(",", ":")
)

hashTxn = hmac.new(
   LLAVE_SECRETA,
   datos.encode("utf-8"),
   hashlib.sha256
).hexdigest()

print("Hash:", hashTxn)
```

https://pythononline.net/

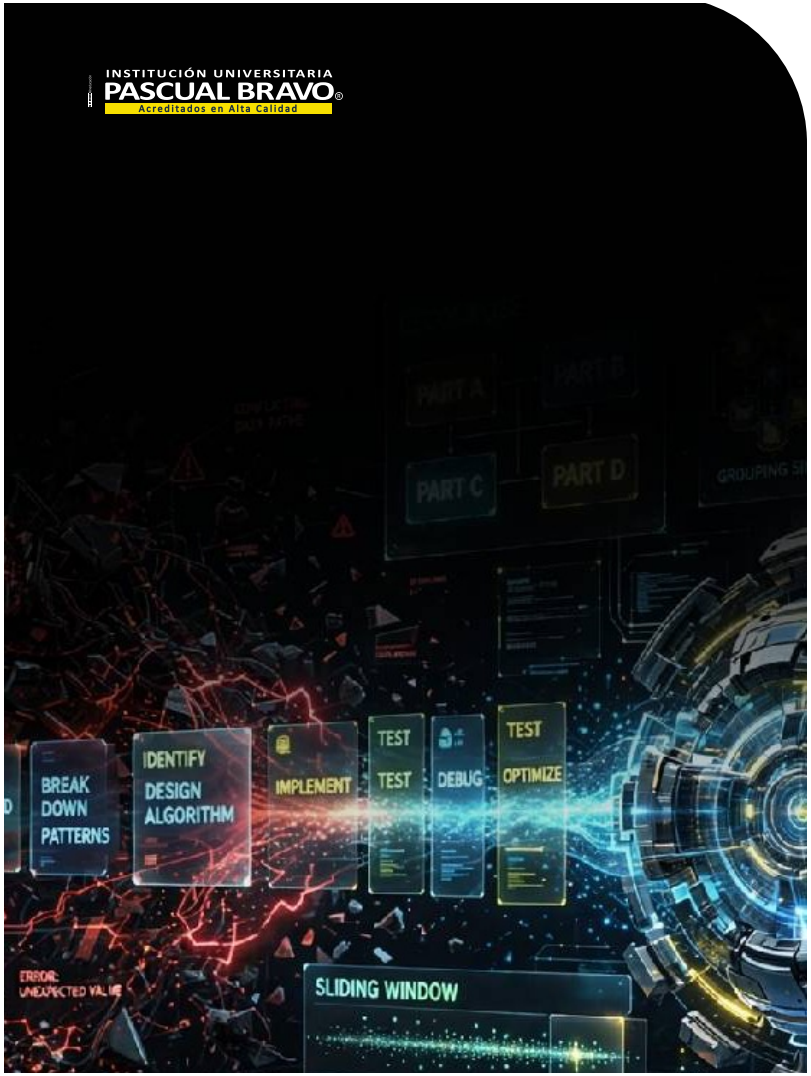
INSTITUCIÓN UNIVERSITARIA
PASCUAL BRAVO
Acreditados en Alta Calidad

Análisis y descomposición de problemas

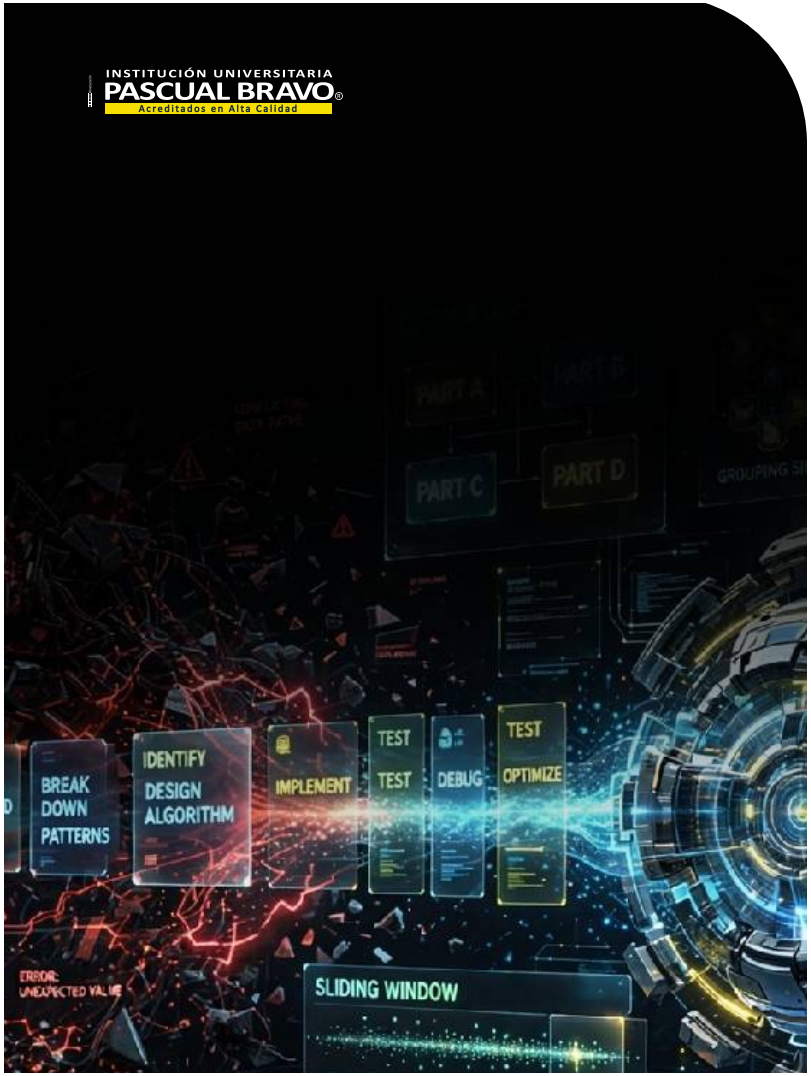


Aprender a convertir un problema grande en problemas pequeños y manejables.

- Identificación del problema.
- Entradas, procesos y salidas.
- División en subproblemas.
- Identificación de restricciones.
- Casos normales y casos límite.




```
productos = [
   {"producto": "Mouse", "valor": "50000", "cantidad": "2"},
   {"producto": "Teclado", "valor": 80000, "cantidad": 1},
]
```




```
productos = [
   {"producto": "Mouse", "valor": "50000", "cantidad": "2"},
   {"producto": "Teclado", "valor": 80000, "cantidad": 1},
]

total = 0

for producto in productos:
   try:
      valor = float(producto["valor"])
      cantidad = int(producto["cantidad"])

      if valor <= 0:
         print(f"Valor inválido para {producto['producto']}")
         continue

      if cantidad <= 0:
         print(f"Cantidad inválida para {producto['producto']}")
         continue

      subtotal = valor * cantidad
      total += subtotal

      print(
         f"{producto['producto']}: "
         f"{cantidad} x ${valor:.0f} = ${subtotal:.0f}"
      )

   except (ValueError, TypeError):
      print(f"Datos inválidos para {producto['producto']}")

print(f"Total: ${total:.0f}")
```

https://pythononline.net/

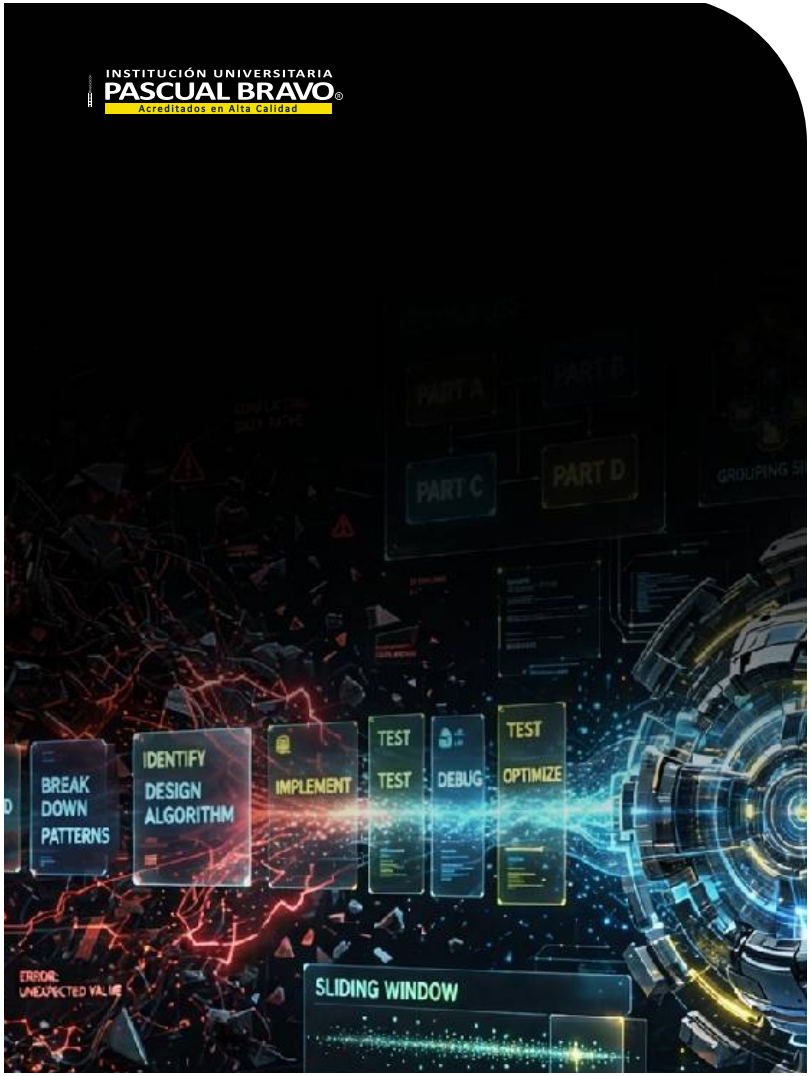
INSTITUCIÓN UNIVERSITARIA
PASCUAL BRAVO
Acreditados en Alta Calidad

Divide y vencerás



Resolver un problema **dividiéndolo** en partes más pequeñas.

- Divide.
- Resuelve.
- Combina.
- Recursividad.



Buscar el número mayor:

```
function encontrarElNumeroMayor(numbers) {
  if (numbers.length === 1)
    return numbers[0];

  const half = Math.floor(numbers.length / 2);

  const left = encontrarElNumeroMayor(numbers.slice(0, half));
  const right = encontrarElNumeroMayor(numbers.slice(half));

  return Math.max(left, right);
}

const data = [10, 5, 30, 8, 20];
const result = encontrarElNumeroMayor(data);
console.log(result);
```

https://www.programiz.com/javascript/online-compiler/

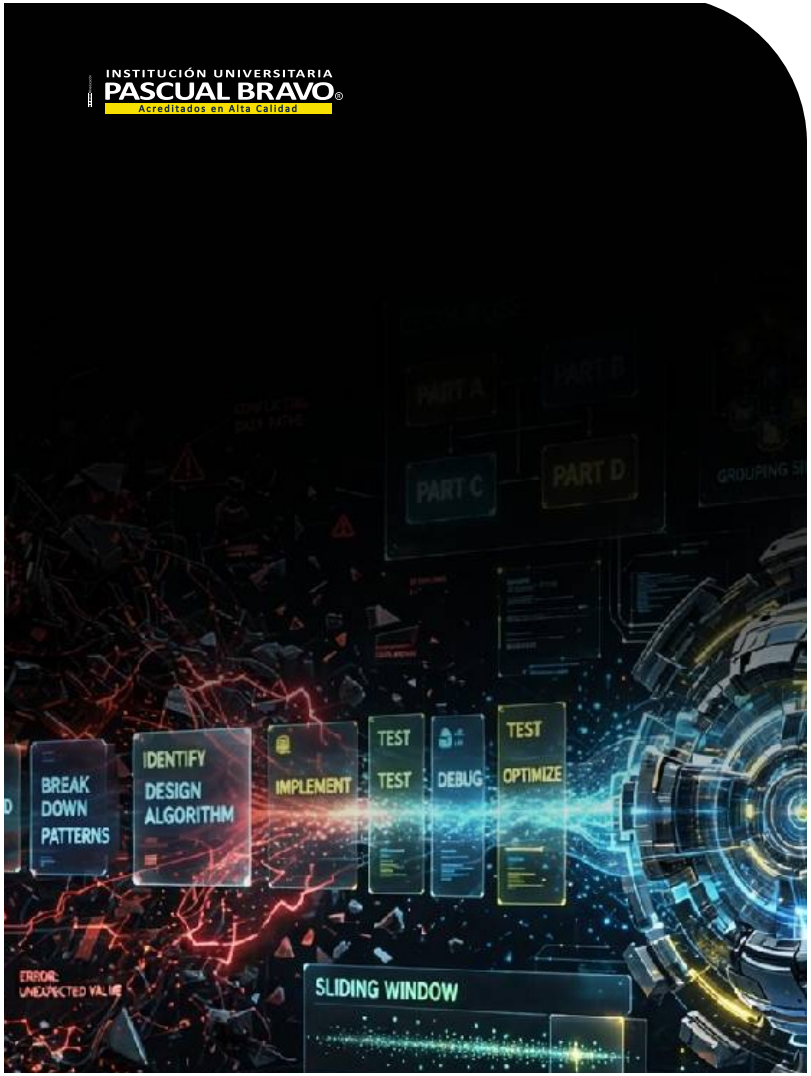
INSTITUCIÓN UNIVERSITARIA
PASCUAL BRAVO
Acreditados en Alta Calidad

Búsqueda y filtrado eficiente



La idea principal es encontrar rápidamente los datos que necesitamos y descartar los que no cumplen una condición, evitando recorrer o procesar información innecesariamente.

- Búsqueda lineal.
- Búsqueda binaria.
- find().
- filter().
- includes().
- Diccionarios/objetos.




```
const usuarios = [
  { id: 1, nombre: "Ana" },
  { id: 2, nombre: "Carlos" },
  { id: 3, nombre: "Pedro" }
];

const usuario = usuarios.find(
  usuario => usuario.id === 2
);
console.log(usuario);
```

```
usuarios = ["Ana", "Carlos", "Pedro", "Laura"]

if "Pedro" in usuarios:
    print("Usuario encontrado")
else:
  print("Usuario NO encontrado")
```

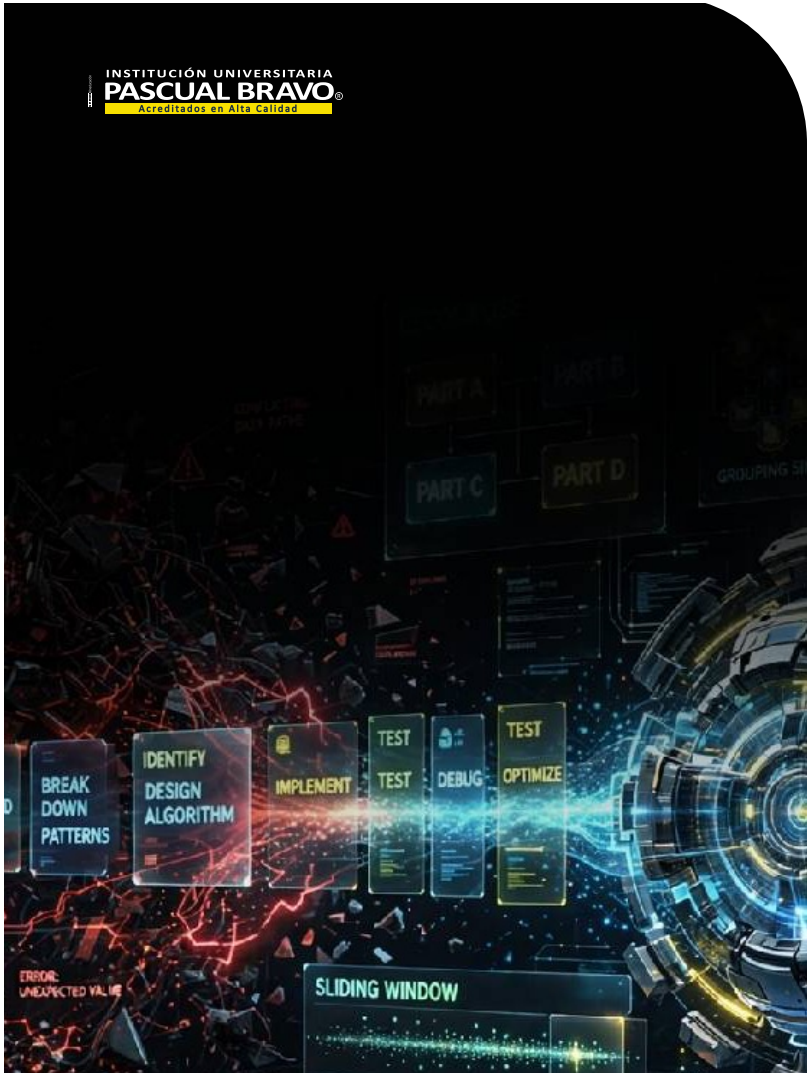
https://www.programiz.com/javascript/online-compiler/
https://www.programiz.com/python-programming/online-compiler/

INSTITUCIÓN UNIVERSITARIA
PASCUAL BRAVO
Acreditados en Alta Calidad

Logs y diagnóstico de problemas



El log debe ayudar a responder qué ocurrió, cuándo ocurrió y dónde ocurrió.




```
def dividir(a, b):
    try:
        return a / b

    except ZeroDivisionError as error:
        with open("app.log", "a") as archivo:
            archivo.write("Error intentando dividir por cero\n")

        return None


resultado = dividir(10, 0)

print("Resultado:", resultado);
```

https://pythononline.net/

INSTITUCIÓN UNIVERSITARIA
PASCUAL BRAVO
Acreditados en Alta Calidad

Ventana Deslizante (Sliding Window)



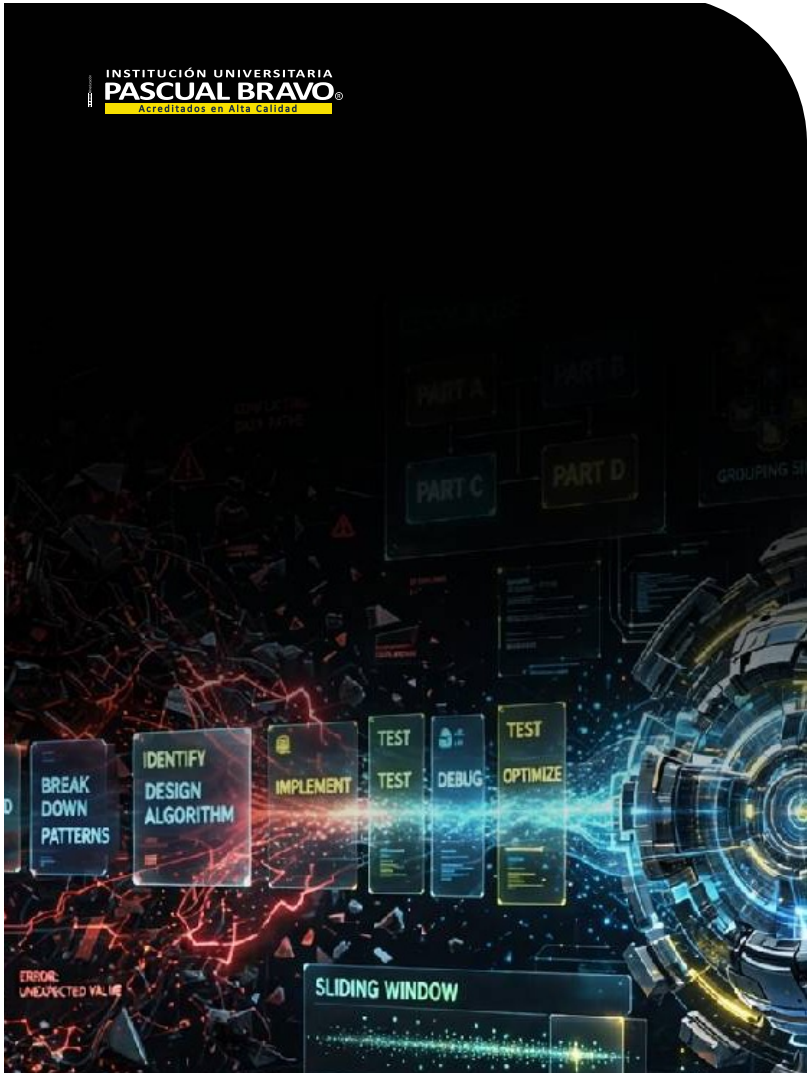
Ventana deslizante, es una técnica de programación que consiste en **analizar una parte** de una colección de datos **a la vez** y mover esa parte **progresivamente**.

[2, 4, 1, 5, 3, 7, 2]

La ventana se mueve:
[2, 4, 1, 5, 3, 7, 2]

Después:
[2, 4, 1, 5, 3, 7, 2]

En lugar de procesar todos los datos nuevamente, reutilizamos información de la ventana anterior, mientras la desplazamos.



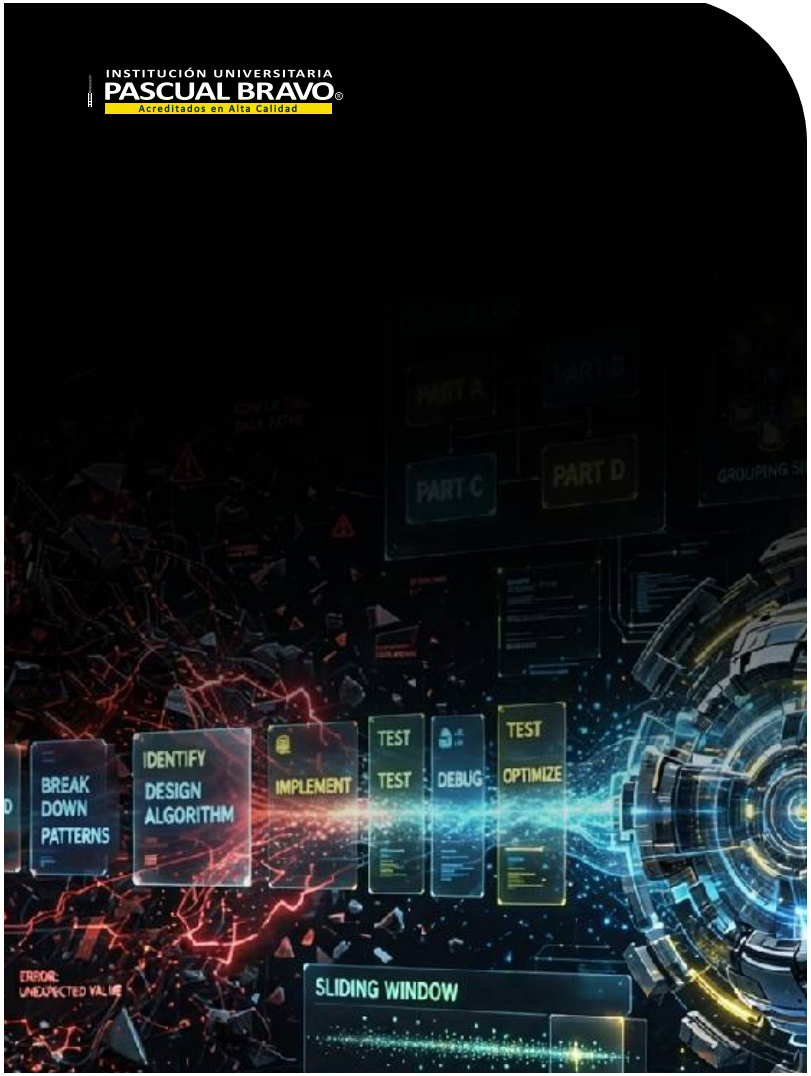
Aplicación en programación

Es útil cuando necesitamos trabajar con elementos consecutivos.

Por ejemplo:

- Promedio de los últimos 7 días
- Análisis de datos en tiempo real, puede utilizarse para analizar:
  - Solicitudes a un servidor
  - Usuarios conectados
  - Temperatura
  - Tráfico
  - Entre otros
- Solicitudes por segundo

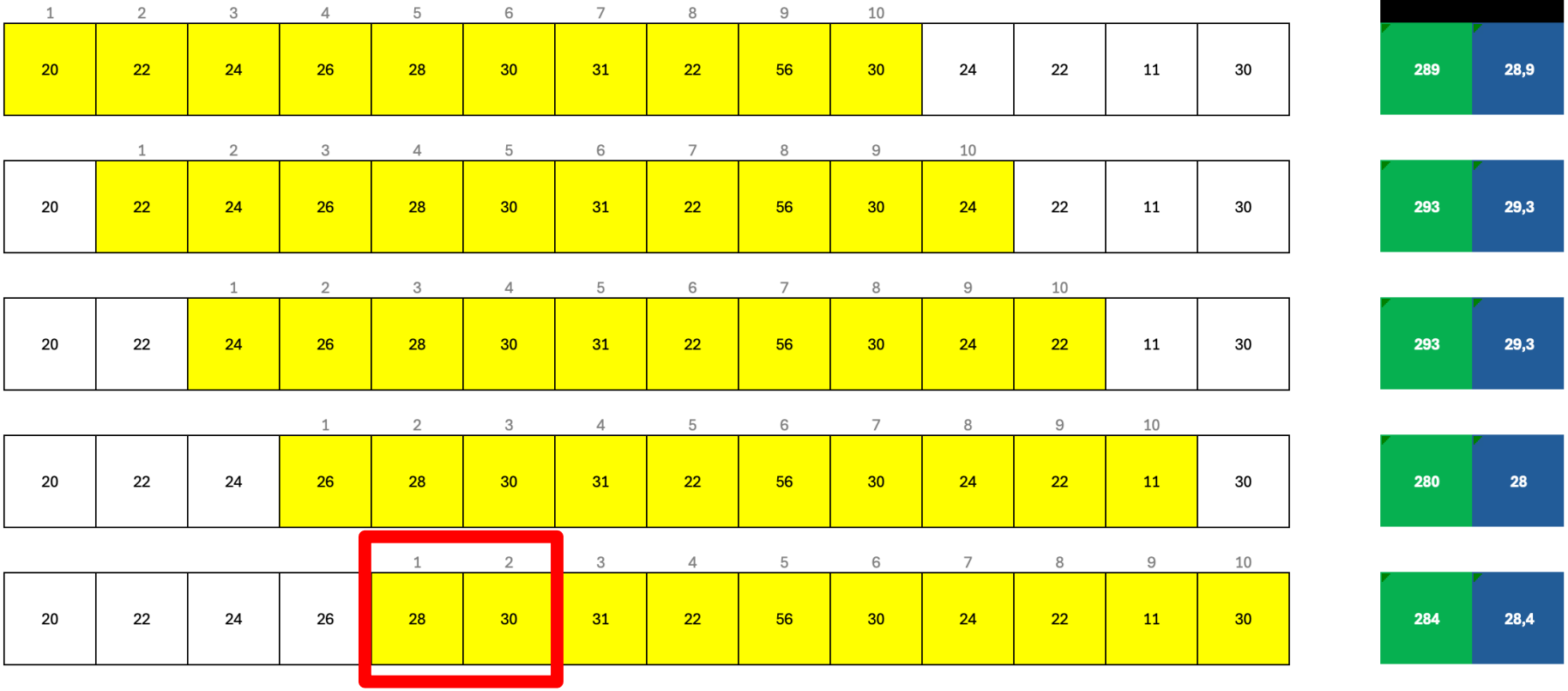
  10 12 15 20 18 25 30



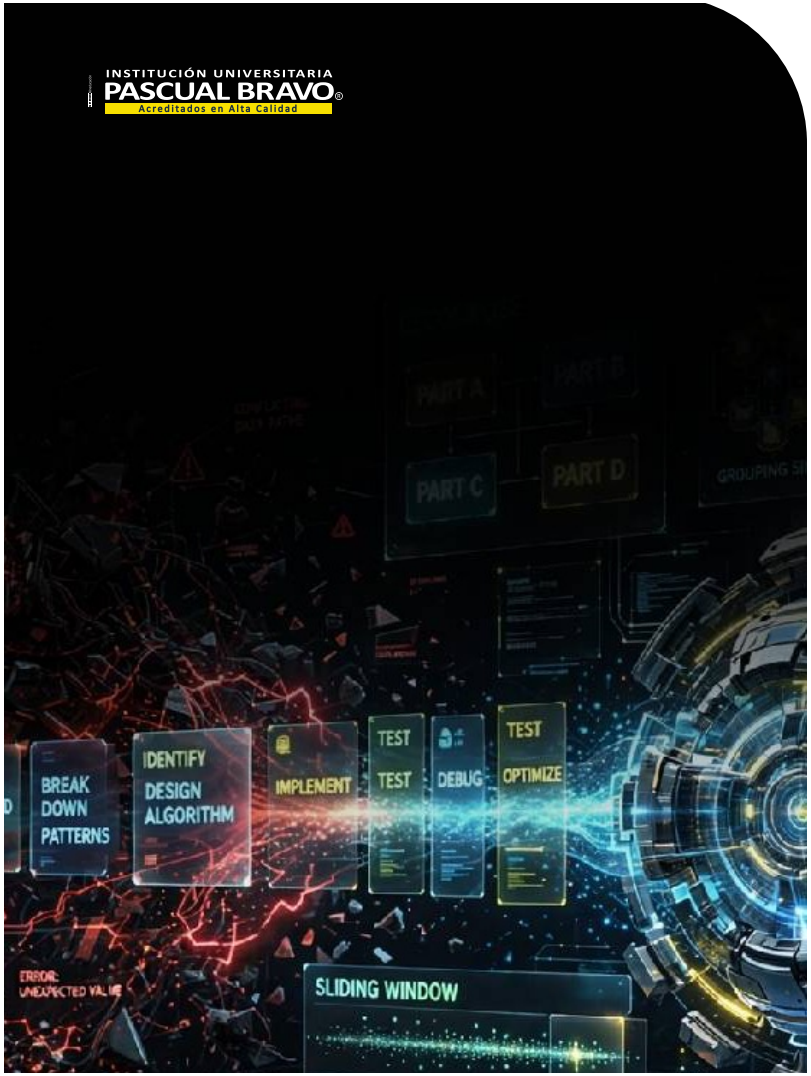
Ventajas

- **Puede mejorar el rendimiento**
  Una de las principales ventajas es evitar cálculos repetidos.
- **Reduce trabajo innecesario**
  SALE un elemento, ENTRA un elemento
- **Es muy útil para datos consecutivos**
  Es especialmente interesante para problemas como:
    Últimos 5 registros
    Últimos 7 días
    Últimas 10 mediciones
    Últimos 30 segundos
- **Es una técnica muy utilizada en algoritmos**
  Aparece frecuentemente en problemas de:
    Arrays
    Strings
    Búsqueda
    Estadísticas
    Procesamiento de datos
    Algoritmos
    Análisis de series temporales

Temperatura - ventana de 10 segundos



SUM	AVG
289	28,9
293	29,3
293	29,3
280	28
284	28,4




```
function promedios(temperaturas, k) {
 let suma = 0;

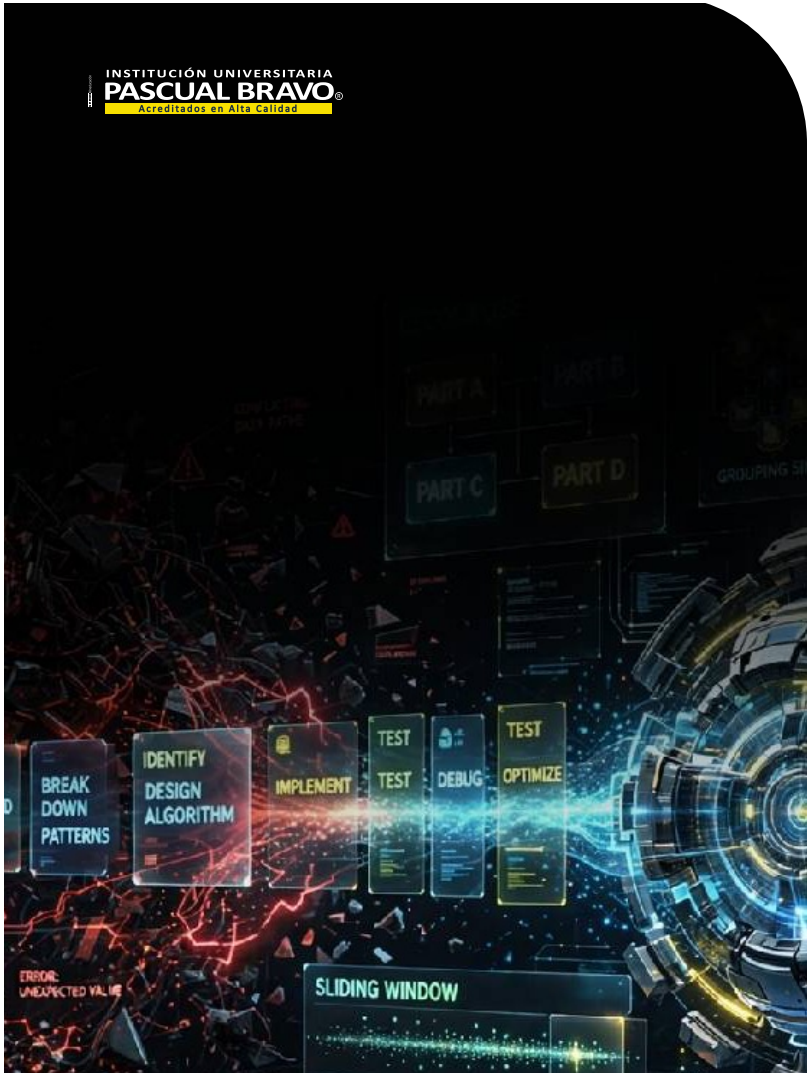
 // Primera ventana
 for (let i = 0; i < k; i++) {
  suma += temperaturas[i];
 }
 let promedio = suma / k;

 console.log(`Primera ventana - SUMA: ${suma} - PROMEDIO: ${promedio}\n\n`);

 // Deslizar la ventana
 for (let i = k; i < temperaturas.length; i++) {
  suma += temperaturas[i];
  suma -= temperaturas[i - k];
 // agrega el elemento nuevo y elimina el elemento que salió.
  promedio = suma / k;
  console.log(`Nueva ventana - SUMA: ${suma} - PROMEDIO: ${promedio}\n\n`);
 }
}
const datos = [20, 22, 24, 26, 28, 30, 31, 22, 56, 30, 24, 22, 11, 30];
const ventana = 10;
promedios(datos, ventana);
```

https://www.programiz.com/javascript/online-compiler/

INSTITUCIÓN UNIVERSITARIA
PASCUAL BRAVO
Acreditados en Alta Calidad




```
function promedios(temperaturas, k) {
  let suma = 0;

  // Primera ventana
  for (let i = 0; i < k; i++) {
    suma += temperaturas[i];
  }
  let promedio = suma / k;

  console.log(`Primera ventana - SUMA: ${suma} - PROMEDIO: ${promedio}\n\n`);

  // Deslizar la ventana
  for (let i = k; i < temperaturas.length; i++) {
    //console.log(`Temperatura: ${temperaturas[i]}`);
    //console.log(`ACUMULADO EN SUMA: ${suma}, SE AGREGA: ${temperaturas[i]}, QUEDANDO ${suma +=
temperaturas[i]}`);
    suma += temperaturas[i];
    //console.log(`SE RESTA A SUMA: ${temperaturas[i - k]}\n\n`);;
    suma -= temperaturas[i - k];
    promedio = suma / k;
    console.log(`Nueva ventana - SUMA: ${suma} - PROMEDIO: ${promedio}\n\n`);
    //console.log("Nueva ventana:", suma / k);
  }
}
const datos = [20, 22, 24, 26, 28, 30, 31, 22, 56, 30, 24, 22, 11, 30];
//console.log(`Largo: ${datos.length}`);
const ventana = 10;
promedios(datos, ventana);
```

https://www.programiz.com/javascript/online-compiler/

Appresso (Tu café, a un tap)

Contexto:

Appresso procesa miles de transacciones diariamente, cada transacción contiene información como:

- ID de transacción
- Fecha y hora
- Usuario (correo)
- IP
- Monto de la transacción
- Método de pago
- Estado
- Hash

Appresso (Tu café, a un tap)

Una de las reglas de detección será:
Si una misma persona realiza múltiples transacciones dentro de una ventana de x(configurable) segundos, el sistema debe marcar el comportamiento como potencialmente sospechoso.

Por ejemplo:
10:30:01 → a@a.com → $50.000
10:30:02 → a@a.com → $30.000
10:30:03 → a@a.com → $20.000
Las tres transacciones ocurrieron dentro de una ventana de aproximadamente 3 segundos.

El sistema debe generar una anomalía: POSIBLE_FRAUDE

Appresso (Tu café, a un tap)

Objetivo
Implementar un sistema de detección de anomalías utilizando la técnica de Ventana Deslizante.

El sistema deberá:

- Recibir transacciones vía POST para ello es necesario construir un endpoint POST el cual recibirá las peticiones.
- Ordenarlas cronológicamente.
- Analizar las transacciones de cada usuario.
- Mantener una ventana temporal de x(configurable) segundos.
- Contar cuántas transacciones existen dentro de la ventana.
- Detectar comportamientos que superen el límite establecido.
- Validar el hash de cada transacción
- Registrar la(s) anomalía(s).
- Identificar usuarios recurrentes.
- Generar estadísticas.
- Mostrar la información en un dashboard.

Appresso (Tu café, a un tap)

El sistema deberá (configurable)

- Por la mañana una **venta de 10**
  entre las 05:00:01 a.m. a 12:00:00 m

- Por la tarde-noche una **venta de 6**
  entre las 12:00:01 m a 08:00 p.m.

- Por la noche-madrugada del siguiente día una **venta de 3**
  entre las 08:00:01 p.m. a 05:00:00 a.m.

INSTITUCIÓN UNIVERSITARIA PASCUAL BRAVO
Acreditados en Alta Calidad

Formar Servir Cuidar

Appresso (Tu café, a un tap)

Regla principal
Para el ejercicio podemos establecer:
3 o más transacciones del mismo usuario dentro de 3 segundos → posible anomalía.

Ejemplo:
Usuario: b@b.com

10:00:01
10:00:02
10:00:03

⚠ POSIBLE ANOMALÍA

10:00:01
10:00:05
10:00:09
✓ Comportamiento normal

Porque las transacciones **no se concentran** dentro de la **ventana de 3 segundos**.

Appresso (Tu café, a un tap)

Lo esperado

Desarrollar una solución capaz de recibir datos **como**:


```
{
  "idTxn": 10001,
  "user": "aa@aa.com",
  "date": "2026-09-23T10:30:01.120",
  "value": 50000,
  "paymentMethod": "Tarjeta”,
  "hash" : "ec37a3a3e8e2566a6ae41d5c807d11db5be922a231d…..",
}
```

Appresso (Tu café, a un tap)

Algoritmo esperado

Recibir transacción
↓
Identificar usuario
↓
Agregar transacción a ventana
↓
Eliminar transacciones fuera de los 3 segundos
↓
Contar transacciones
↓
¿Ejemplo: cantidad >= 3?
/ \
SI NO
↓ ↓
Anomalía Normal
↓ ↓
Registrar Registrar

INSTITUCIÓN UNIVERSITARIA PASCUAL BRAVO
Acreditados en Alta Calidad

Formar Servir Cuidar

Appresso (Tu café, a un tap)

Casos de uso

Caso de uso 1 — Usuario realiza múltiples transacciones
Usuario: b@b.com

10:00:01
10:00:02
10:00:03 .
Resultado ANOMALÍA

Caso de uso 2 — Transacciones normales
Usuario: c@c.com

10:00:01
10:00:10
10:01:20
Resultado NORMAL

Appresso (Tu café, a un tap)

Caso de uso 3 — Diferentes usuarios

Usuario 1 → 10:00:01
Usuario 2 → 10:00:02
Usuario 3 → 10:00:03

No se deben mezclar las ventanas, cada usuario tiene su propia ventana:
Usuario 1 → [T1] .
Usuario 2 → [T2]
Usuario 3 → [T3]

Appresso (Tu café, a un tap)

Dashboard
El dashboard debería permitir visualizar información agregada.
Hoy (10) | Esta semana (50) | Este mes (127)

- Casos más recurrentes
- Múltiples transacciones
- Usuarios recurrentes
- Total de transacciones (día, semana, mes)
- Total de anomalías (día, semana, mes)
- Porcentaje de transacciones con anomalías.
- Usuarios afectados.
- Valor total de transacciones sospechosas.
- Promedio de transacciones por usuario.
- Número de anomalías nuevas.
- Anomalías abiertas.
- Anomalías revisadas.
- Anomalías descartadas.
- Tendencias

- Evolución temporal, permite identificar:
- Horas con más anomalías.
- Picos repentinos.
- Periodos de actividad.
- Tendencias.
- Anomalías por nivel
- **Línea de tiempo de una anomalía**
- Visualización de la ventana deslizante
- Métodos de pago
- Distribución por hora (intensidad representa según la cantidad de anomalías x horas)

INSTITUCIÓN UNIVERSITARIA PASCUAL BRAVO
Acreditados en Alta Calidad

Formar Servir Cuidar

Appresso (Tu café, a un tap)

usuarios
id PK
nombre
email
estado
fecha_creación
fecha_actualizacion

transacciones
id PK
usuario_id
valor
fecha_txn
estado
hash
metodo_pago
fecha_creación
fecha_actualizacion

anomalias
id PK
transaccion_id
tipo
nivel
cantidad_transacciones
ventana_segundos
fecha_creación
fecha_actualizacion